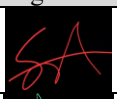
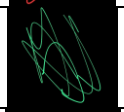
Project Software Project Management Plan (SPMP) Form

Project Title:

InternSync – Cooperative Training Coordination and Tracking System

Team Information:

Team Members	ID	Signature	Date
Salam Alsaour (Team Leader)	2250009168		Oct 3, 2026
Khalid Alajmi	2250004309		Oct 3, 2026
Ali Alghafly	2250002460		Oct 3, 2026
Ahmed Kanaan	2250009170		Oct 3, 2026
Abdullah Abalhareth	2250005266		Oct 3, 2026
Rakan Alzahrani	2250000631		Oct 3, 2026

Public - عام

Project organization:

Explain how the project is organized and how the team is structured.

Process Model

Describe the development approach and why you chose it.

- **Plan-driven development with incremental delivery.** All activities (requirements, design, implementation, testing, documentation) are planned in advance with dated phases and deliverables, and progress is measured against the plan.
- **Why:** the course fixes the deadlines and the documents to be submitted, so requirements, design, testing, and documentation follow dated phases. An agile process such as Scrum relies on flexible time and scope, which does not match a fixed schedule with graded deliverables.
- **Incremental delivery:** implementation is split into three planned increments ordered by priority (Must, then Should, then Could), so a working, tested system exists early. If the schedule slips, lower-priority functions are dropped instead of moving the deadlines. A strictly sequential Waterfall process was rejected because it leaves testing until the end.
- **Change control:** after the SRS is approved, requirement changes are recorded and accepted only if they fit the schedule.
- **Quality practices:** Git feature branches and pull requests with at least one reviewer; automated tests run on every pull request; documentation (SRS, design, test report) is written alongside the work.

Organizational Structure

Describe how the team is organized and how authority and reporting lines work.

- **Team Leader (Salam Alsaour)** coordinates all work, maintains the schedule, and reports to the course supervisor.
- **Five sub-teams:** Frontend/PWA (Ali, Khalid); Backend and Database (Salam, Khalid); AI services (Salam, Ahmed), with Ahmed also responsible for deployment; UI/UX design (Rakan); QA and Documentation (Abdullah). Members may support other sub-teams when needed.
- **Reporting lines:** sub-team members report to the Team Leader; the Team Leader reports to the course supervisor at each milestone.
- **Decisions** are made by team vote at the weekly meeting, and the Team Leader breaks ties. Technical decisions inside a sub-team are made by that sub-team and shared with the whole team.

Team roles and responsibilities:

List each team member and their main responsibilities.

Team Members	Role	Responsibilities
Salam Alsaour	Team Leader; backend and AI services developer	Plans and tracks the schedule, runs the weekly meeting, reports to the course supervisor, integrates the components; develops the matching engine and similarity detection; in the backend: weekly reports, evaluations, scheduled notifications, PDF/Excel export.
Khalid Alajmi	Frontend and backend developer	Backend: database schema, authentication and role-based access control, approval workflow, API for profiles and vacancies. Frontend: screens for reports, evaluations, and the public certificate page.
Ali Alghafly	Frontend developer	React PWA: role dashboards, forms, Arabic/English interface with RTL support, notifications on the client.
Ahmed Kanaan	AI services developer; deployment lead	Skill extraction from CVs, match explanation and development plan (working with Salam on the AI services); deployment of the system (Docker and the demo server).
Abdullah Abalhareth	QA and documentation lead	Test plan and test cases, user acceptance testing, SRS and design documents, keeps the task board and the risk register up to date.
Rakan Alzahrani	UI/UX designer	Wireframes and design system in Claude Design + Figma Make, usability of key screens.

Activities and Tasks:

List the main activities that need to be completed.

Main activities:

- Plan the project and collect requirements (SRS)
- Design the system architecture, database (ERD), and API
- Design the user interface (Claude Design + Figma Make wireframes, Arabic/English, PWA)
- Develop the backend (authentication, vacancies, reports, evaluations, approval workflow, notifications)
- Develop the frontend (role dashboards, forms, PWA features)
- Develop the AI services (matching, explanation and development plan, similarity detection)
- Integrate components and export grade sheets (PDF and Excel)
- Test the system (unit, integration, and user acceptance testing) and fix defects
- Prepare final documentation and presentation
- Manage the project (weekly meetings, tracking, change and risk control)

Schedule:

Show when each activity starts and ends and how they depend on each other.

ID	Activity	Start	End	Duration	Depends On	Owner
T1	Plan project and collect requirements (SRS)	Sep 25	Oct 3	9 days	-	Salam, Abdullah
T2	System design (architecture, ERD, API specification)	Oct 4	Oct 15	12 days	T1	Khalid, Ahmed, Salam
T3	UI/UX design (Claude Design + Figma Make wireframes)	Oct 4	Oct 15	12 days	T1	Rakan, Ali
T4	Increment 1: authentication, profiles and CV upload, vacancies, weekly reports, bilingual interface foundation	Oct 16	Nov 1	17 days	T2, T3	All developers
T5	Increment 2: matching, attendance, evaluations, approval workflow, dashboards (all Must functions)	Nov 2	Nov 15	14 days	T4	All developers
T6	Increment 3: explanation and skill gaps, similarity detection, notifications, exports, PWA (Should functions)	Nov 16	Nov 25	10 days	T5	All developers
T7	Testing and bug fixing (unit, integration, user acceptance); Could functions if time allows	Nov 26	Dec 10	15 days	T6	Abdullah, all
T8	Final documentation and presentation	Dec 11	Dec 15	5 days	T7	Abdullah, Salam
T9	Project management and monitoring (weekly meetings, tracking, change and risk control)	Sep 25	Dec 15	82 days	-	Salam

All dates are in 2026.

Milestones:

List the key checkpoints, each with a date and a clear completion criterion.

Milestone	Deliverable	Target Date	Status
M1: Requirements baseline	SRS approved by the team and requirements frozen	Oct 3	In progress
M2: Design complete	Architecture, ERD, API specification, and Claude Design + Figma Make wireframes reviewed and approved	Oct 15	Planned
M3: Increment 1 delivered	Demo of F1, F2, F3, F5, F10 foundation; automated tests pass	Nov 1	Planned
M4: Increment 2 delivered (all Must functions)	Demo of F4, F6, F7, F8, F9; every Must function passes its tests	Nov 15	Planned
M5: Increment 3 delivered	Demo of F11–F15; Should functions pass their tests	Nov 25	Planned
M6: Testing complete	Test report; user acceptance testing with at least 8 participants, 80% or more of scenarios completed without help	Dec 10	Planned
M7: Final delivery	Final system, documentation, and presentation	Dec 15	Planned

Resource planning:

List the resources needed to complete the project.

People:

- Team Leader / project manager (1)
- Backend and database developers (2)
- AI services developers (2, one shared with the backend)
- Frontend developers (2, one shared with the backend)
- UI/UX designer (1)
- Deployment lead (1, also an AI services developer)
- QA and documentation lead (1)
- At least 8 volunteer users for user acceptance testing (students, faculty, and company representatives)

Software and tools:

- Python and FastAPI (backend and AI services)
- React with Vite (PWA frontend)
- PostgreSQL with the pgvector extension
- Open-source multilingual embedding model (e.g., multilingual-e5 via sentence-transformers)
- Docker
- GitHub (version control, Projects board, automated tests on pull requests)
- Claude Design + Figma Make (UI/UX design)
- VS Code, pytest, and Postman (development and testing)

Hardware, data, and effort:

- Team laptops for development; one cloud or university server (free or low-cost tier) for the demo, with enough memory to run a small embedding model
- Sample and synthetic data: CVs, vacancies, and reports, including a labelled test set of at least 20 student profiles for matching and 10 seeded copied reports for similarity detection
- Effort: about 8 hours per member per week over 10 weeks ($6 \times 8 \times 10 \approx 480$ person-hours)

Risk identification and management:

Identify what could go wrong, rate it, and say how you will respond.

Risk	Probability	Impact	Mitigation
Schedule slips because of team availability	High	High	MoSCoW priorities; weekly tracking; drop Could functions first, then Should functions.
Late requirement changes and scope creep	Medium	High	Freeze requirements after the SRS; change control; changes accepted only if they fit the schedule.
Team members unfamiliar with FastAPI, React, or pgvector	Medium	Medium	Short learning tasks during the design phase; tasks assigned by skill; code reviews and pair work.
Matching or similarity results are unreliable or hard to explain	Medium	High	Rule-based fallback; show matched and missing skills; advisory use only; test on a labelled sample.
Extracting skills from CVs in different formats	Medium	Medium	Start with PDF and DOCX text; let the student review and edit extracted skills.
Arabic (RTL) text in PDF exports	Medium	Medium	Test export early in Increment 1 with bundled Arabic fonts.
Permission errors expose another user's data	Low	High	Central role-based access control with automated permission tests.
Inconsistent deadline, reminder, or approval states	Medium	Medium	Model approvals as an explicit state machine and test every transition.
Integration problems between frontend, backend, and AI services	Medium	Medium	Agree the API specification first; generate the TypeScript client from the OpenAPI schema; integrate every week.

Project monitoring and control:

Explain how the project progress will be checked and controlled.

Monitoring and control activities:

- **Weekly check:** the team meets every week to compare completed tasks with the schedule (T1–T9) and the task board on GitHub Projects.
- **Track unfinished tasks:** tasks not finished in the planned week are flagged, given a new owner or date, and their effect on the next milestone is assessed.
- **Milestone reviews:** at the end of each increment the team holds a demo, lists defects, and decides which remaining Should and Could functions stay in the plan.
- **Change control:** every change request is recorded with its effect on schedule and scope; the Team Leader approves it after a team vote.
- **Risk review:** the risk table is reviewed every week and updated with new risks and their status.
- **Quality control:** pull-request reviews, automated tests, and a test report before final delivery.
- **Reporting:** the Team Leader reports progress to the course supervisor at each milestone.